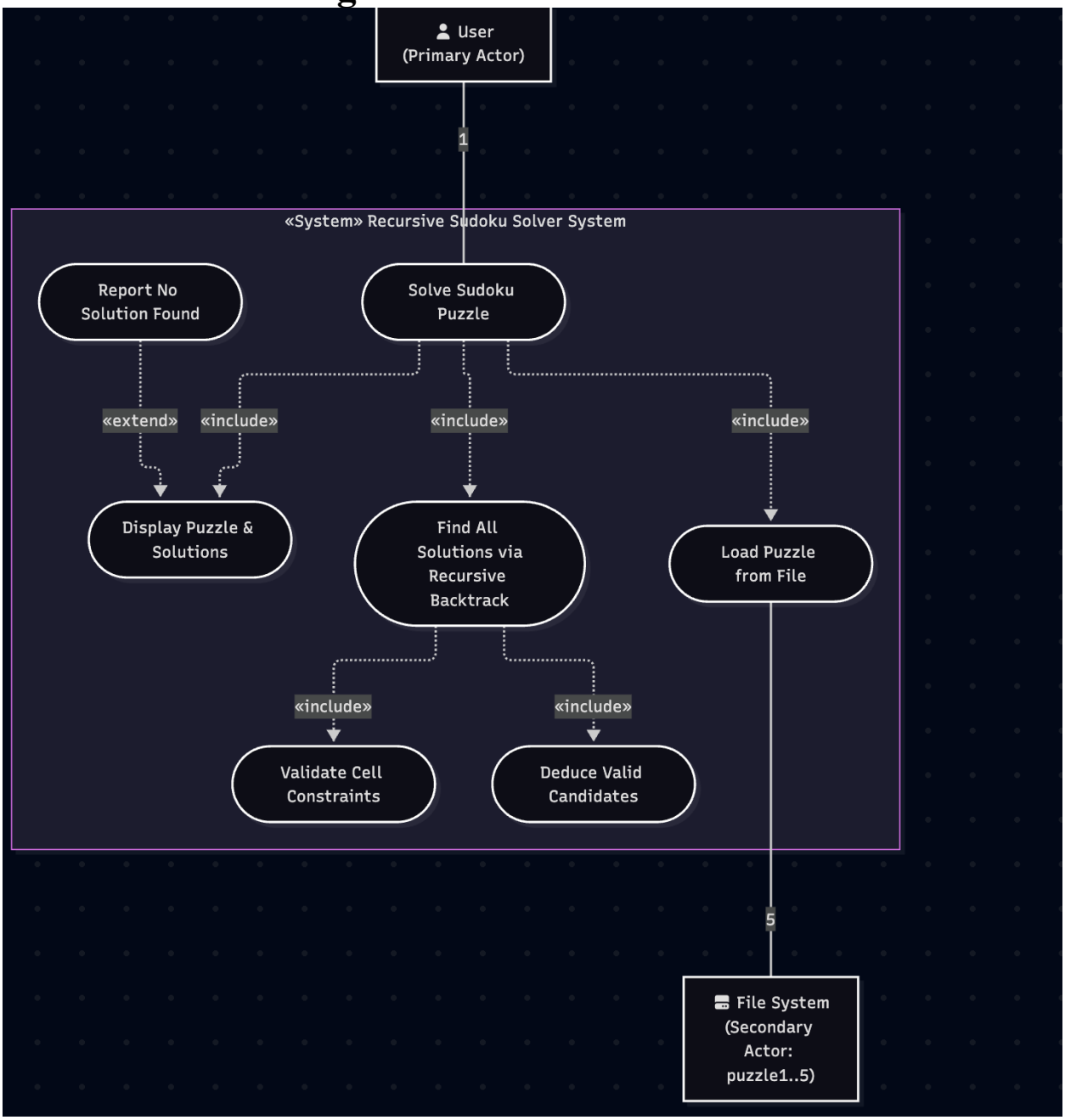
EECS 348 Assignment 4: UML Use Case Model

Student: Joshua Assefa

Course: EECS 348 – Software Engineering I (Fall 2026)

System: Recursive Object-Oriented Sudoku Solver System

UML Use Case Diagram



Step 1: Identify / Discover Use Cases and Actors

Actors

- **Primary Actor: User** (Student / Grader executing the Sudoku Solver application).

- **Secondary / Supporting Actor: File System** (Provides input files puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt).

Discovered Use Cases

1. **Solve Sudoku Puzzle** *(User-Goal Level Base Use Case)*
2. **Load Puzzle from File** *(Included Subfunction Use Case — «include»)*
3. **Find All Solutions via Recursive Backtrack** *(Included Subfunction Use Case — «include»)*
4. **Deduce Valid Candidates** *(Included Subfunction Use Case — «include»)*
5. **Validate Cell Constraints** *(Included Subfunction Use Case — «include»)*
6. **Display Puzzle & Solutions** *(Included Subfunction Use Case — «include»)*
7. **Report No Solution Found** *(Extending Use Case — «extend» when solutionCount == 0)*

Step 2: Provide a Brief Description for the Use Cases

- **Solve Sudoku Puzzle:** This use case allows the User to automatically load, display, and solve the five partially completed 9×9 Sudoku puzzle files (puzzle1.txt through puzzle5.txt), outputting every valid solution or proving that no solution exists.
- **Load Puzzle from File:** Reads a 9×9 grid of characters (1–9 and _) from a puzzle text file in the File System into a SudokuBoard object.
- **Find All Solutions via Recursive Backtrack:** Executes depth-first search with backtracking across all unfilled cells (k empty cells) to discover every valid completion of the 9×9 grid.
- **Deduce Valid Candidates:** Identifies all digits from 1 to 9 that can legally be placed into a specific empty cell without violating row, column, or 3×3 block rules.
- **Validate Cell Constraints:** Verifies that a candidate digit does not already appear in the target cell's row, column, or 3×3 sub-grid.
- **Display Puzzle & Solutions:** Prints the puzzle filename, the initial partially filled grid, and each solved 9×9 grid to standard output.
- **Report No Solution Found:** Extends the display behavior by printing "No solution found" when the exhaustive recursive search completes with zero valid solutions.

Step 3: Provide a Step-by-Step Flow (Outline) of Events

Outline for Base Use Case: *Solve Sudoku Puzzle*

1. User launches the Sudoku Solver program.
2. System selects the next puzzle file (puzzle1.txt through puzzle5.txt).
3. System opens and reads the 9×9 puzzle grid from the File System (Load Puzzle from File).

4. System prints the puzzle file name and the initial 9×9 puzzle grid (Display Puzzle & Solutions).
5. System validates the initial pre-filled board constraints (Validate Cell Constraints).
6. System locates the next empty cell (_) in the grid.
7. System deduces the list of legal candidate digits (1–9) for the empty cell (Deduce Valid Candidates).
8. System recursively places each candidate digit and advances to the next empty cell (Find All Solutions via Recursive Backtrack).
9. Whenever all 81 cells are validly filled, System prints the solved grid and backtracks to find any remaining solutions.
10. If all candidate paths are exhausted and zero solutions were found, System prints "No solution found" (Report No Solution Found).
11. System repeats steps 2–10 until all five puzzle files have been processed.

Step 4: Provide a Detailed Description for the Use Case

Use Case: Solve Sudoku Puzzle

- **Primary Actor:** User
- **Secondary Actor:** File System (puzzle1.txt – puzzle5.txt)
- **Goal:** Find and display all valid solutions for each of the five 9×9 Sudoku puzzle files, or prove that no solution exists by exhausting all candidate paths.
- **Preconditions:**
    1. The executable Assignment4.exe is compiled and invoked in a directory containing puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt.
    2. Each puzzle file contains 81 whitespace-separated tokens (1–9 or _).
- **Trigger:** User executes ./Assignment4.exe from the command line.

1. Basic Flow (Main Success Scenario)

1. **Initiate Execution:**
    - The User starts the application.
    - The System instantiates a SudokuSolver object and iterates sequentially through the five filenames (puzzle1.txt to puzzle5.txt).
2. **Load and Display Initial Puzzle («include» Load Puzzle from File):**
    - The System prints the current file name (e.g., puzzle1.txt).
    - The System opens the file stream, reads all 81 tokens, converts '1'–'9' to integers 1–9 and '_' to 0, and populates the SudokuBoard object.
    - The System outputs the initial 9×9 puzzle grid to standard output.
3. **Locate Unfilled Cell («include» Find All Solutions via Recursive Backtrack):**
    - The System scans the 9×9 grid row-by-row from top-left (0,0) to bottom-right (8,8) to locate the first empty cell containing 0.
    - If no empty cell exists ($k = 81$ filled cells), the System increments solutionCount, prints the completed 9×9 solution grid to standard output,

and returns to the caller frame to continue searching for additional solutions.

4. **Deduce and Test Candidates («include» Deduce Valid Candidates & Validate Cell Constraints):**
    - For the identified empty cell (row, col), the System tests digits 1 through 9 against row, column, and 3×3 block constraints.
    - The System constructs a candidate list of all conflict-free digits.
    - For each candidate in the list, the System writes the digit into grid[row][col] and recursively invokes step 3 (solveRecursive) with $k + 1$ cells filled.
    - Upon returning from the recursive call, the System resets grid[row][col] = 0 (backtracking) and advances to the next candidate in the list.
5. **Conclude Puzzle Processing («include» Display Puzzle & Solutions):**
    - Once the root recursive call finishes exploring all branches, the System checks solutionCount.
    - Since solutionCount >= 1 (e.g., 1 solution for puzzle1.txt and puzzle5.txt; 2 solutions for puzzle3.txt), the System prints the total number of solutions found and proceeds to the next puzzle file.

2. Alternative / Exception Flows

- **Alternative Flow A1: No Valid Candidates for an Empty Cell (Dead End / Backtrack)**
    1. At Step 4 of the Basic Flow, getCandidates(row, col) returns an empty list because all digits 1–9 already appear in the cell's row, column, or 3×3 block.
    2. The System recognizes that an earlier tentative assignment was invalid, immediately returns from the current recursive call, restores the parent cell to 0, and tries the next candidate in the parent frame.
- **Alternative Flow A2: Unsolvable Puzzle («extend» Report No Solution Found — e.g., puzzle2.txt, puzzle4.txt)**
    1. At Step 5 of the Basic Flow, the exhaustive backtracking search completes (or isInitialBoardValid() fails), and solutionCount == 0.
    2. The Report No Solution Found extension executes and outputs "No solution found" to standard output.
    3. The System proceeds to the next puzzle file.
- **Alternative Flow A3: Puzzle File Missing or Unreadable**
    1. At Step 2 of the Basic Flow, loadFromFile() fails to open the target .txt file.
    2. The System outputs an error message indicating the file could not be read and advances to the next file in the list.

3. Postconditions

- All five puzzle files have been displayed and evaluated, with every valid solution (or "No solution found") printed to standard output.